CAPSTONE PROJECT THREE

ASL DETECTION AND CLASSIFICATION

PROJECT REPORT

Submitted by	Prativa Kunwar
Student ID	_____
Course / Subject	AI/ML
Institution	Women in Animation
Date of Submission	November 1, 2026

Women in Animation

ABSTRACT

This project investigates an American Sign Language (ASL) hand-gesture recognition system using deep learning. The classification component recognizes 36 classes: digits 0–9 and lowercase alphabet labels a–z. Three pretrained convolutional neural network architectures—ResNet18, VGG16, and AlexNet—are adapted through transfer learning and compared using recorded accuracy, loss, and training time. A custom CNN is also designed and evaluated.

The custom CNN uses two convolutional blocks followed by fully connected layers. The recorded experiment reports 92.48% validation accuracy and 94.44% test accuracy. Five-fold cross-validation reports an average best validation accuracy of 90.06%. The detection component uses YOLO to locate hands as a single object class. A detected hand crop can then be passed to the CNN classifier to predict the corresponding digit or alphabet class.

The recorded transfer-learning results are preliminary because the original training loop used the test set during epoch-wise evaluation. Those experiments should be rerun with validation data used during training and the test set reserved for final evaluation. Final one-class YOLO metrics and a verified end-to-end result were not available in the supplied project documentation and are therefore not invented in this report.

Keywords: American Sign Language, CNN, transfer learning, ResNet18, VGG16, AlexNet, YOLO, image classification, object detection.

TABLE OF CONTENTS

CHAPTER 1 · INTRODUCTION 4

1.1 Background and Overview 4
1.2 Problem Statement 4
1.3 Objectives 4
1.4 Scope 5

CHAPTER 2 · SYSTEM OVERVIEW 5

2.1 Overall Workflow 5
2.2 Classification Pipeline 5
2.3 Detection and Classification Pipeline 5

CHAPTER 3 · DATASET AND METHODOLOGY 6

3.1 Dataset Description 6
3.2 Image Preprocessing 6
3.3 Data Augmentation 6
3.4 Transfer Learning 6
3.5 Custom CNN Architecture 6
3.6 Parameter Calculation 8
3.7 YOLO Hand Detection 8

CHAPTER 4 · IMPLEMENTATION AND TRAINING 9

4.1 Tools and Execution Environment 9
4.2 Classification Training 9
4.3 Custom CNN Training 9
4.4 Five-Fold Cross-Validation 10
4.5 YOLO Training and Integration 10

CHAPTER 5 · RESULTS AND DISCUSSION 11

5.1 Transfer-Learning Comparison 11
5.2 Custom CNN Results 11
5.3 Cross-Validation Results 11
5.4 YOLO Detection Results 12
5.5 Discussion and Limitations 12

CONCLUSION 13

REFERENCES 14

LIST OF FIGURES

Figure 1. Integrated hand detection and classification pipeline

LIST OF TABLES

Table 1. Classification dataset split

Table 2. Data augmentation techniques

Table 3. Custom CNN architecture

Table 4. Custom CNN trainable parameter calculation

Table 5. Transfer-learning model comparison

Table 6. Custom CNN evaluation results

Table 7. Five-fold cross-validation results

Table 8. YOLO detection metrics status

LIST OF ABBREVIATIONS

Abbreviation	Full form
ASL	American Sign Language
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DL	Deep Learning
FC	Fully Connected
GPU	Graphics Processing Unit
IoU	Intersection over Union
mAP	Mean Average Precision
ReLU	Rectified Linear Unit
RGB	Red, Green, Blue
YOLO	You Only Look Once

CHAPTER 1 — INTRODUCTION

1.1 Background and Overview

American Sign Language is a visual language in which hand shapes, positions, and movements communicate meaning. Computer vision and deep learning can be used to analyze hand images and recognize gesture categories automatically. Convolutional neural networks are useful for image classification because they learn spatial features such as edges, shapes, textures, and more complex visual patterns.

This project investigates two connected computer-vision tasks: hand detection and ASL gesture classification. YOLO is used to locate the hand in an image, while a CNN predicts which of the 36 target classes the hand gesture represents. Transfer-learning models and a custom CNN are also compared as part of the classification experiments.

1.2 Problem Statement

Hand-gesture recognition is challenging because images can vary in lighting, background, hand orientation, scale, and camera quality. Similar-looking gestures may also be difficult to distinguish. Classifying an entire image can be affected by irrelevant background content or by the hand occupying only a small part of the image.

The project therefore addresses two related problems: (1) locating a hand in an image using object detection, and (2) classifying the hand image into one of 36 gesture classes using a CNN.

1.3 Objectives

• Prepare an image dataset containing ASL digits and alphabet gestures.

• Preprocess images into a consistent format suitable for deep learning.

• Apply at least three data augmentation techniques.

• Implement ResNet18, VGG16, and AlexNet using transfer learning.

• Compare classification models using accuracy, loss, and training time.

• Design a custom CNN and calculate its trainable parameters.

• Train and evaluate the custom CNN, targeting at least 83% classification accuracy.

• Perform five-fold cross-validation and record fold-wise accuracy and loss.

• Train a YOLO model to detect hand regions.

• Investigate integration of hand detection and gesture classification.

1.4 Scope

The classification task contains 36 categories: ten numerical classes (0–9) and 26 lowercase alphabet classes (a–z). The detection model uses a different label structure: one object class, named **hands**, to identify the location of a hand. YOLO does not directly predict the 36 gesture categories; the CNN performs that classification after the hand crop is extracted.

CHAPTER 2 — SYSTEM OVERVIEW

2.1 Overall Workflow

The intended integrated system captures an image or video frame, detects the hand with YOLO, crops the detected region, preprocesses the crop, and passes it to the CNN classifier. The classifier selects the highest-scoring class and displays the predicted digit or alphabet label.

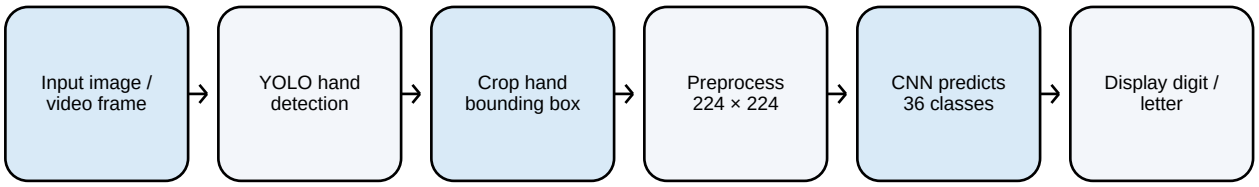

Figure 1. Integrated hand detection and classification pipeline

2.2 Classification Pipeline

• Dataset preparation: images of digits 0–9 and lowercase letters a–z are organized into class-specific folders.

• Preprocessing: images are resized to 224 × 224 pixels, converted to tensors, and normalized.

• Augmentation: random horizontal flipping, rotation, and brightness/contrast adjustment are applied to training images.

• Model training: ResNet18, VGG16, AlexNet, and a custom CNN are trained using the prepared dataset.

• Evaluation: accuracy and loss are recorded; training time is measured for model comparison.

• Cross-validation: the custom CNN is evaluated using five stratified folds.

• Model saving: model weights and the class-to-index mapping are saved for later use.

2.3 Detection and Classification Pipeline

1. Capture or load an input image/frame.

2. Run the trained YOLO model to identify a hand bounding box.

3. Crop the detected hand region using the bounding-box coordinates.

4. Resize the crop to 224 × 224 and apply the classifier's normalization.

5. Pass the crop through the trained CNN.

6. Map the highest-scoring output index to its class label.

7. Display the predicted digit or alphabet label.

> ***Key distinction:*** *YOLO answers “Where is the hand?” The CNN answers “Which gesture does the hand represent?” A single YOLO class and 36 CNN classes are therefore appropriate for the two different tasks.*

CHAPTER 3 — DATASET AND METHODOLOGY

3.1 Dataset Description

The classification dataset contains 2,515 images across 36 classes: digits 0–9 and lowercase letters a–z. The dataset is organized into train, validation, and test folders, with class-specific subfolders.

Subset	Images	Purpose
Training	1,760	Learn model parameters
Validation	359	Monitor performance during training and model selection
Testing	396	Evaluate final generalization
Total	2,515	All three subsets

The dataset is loaded with PyTorch's ImageFolder utility. The class labels are assigned in sorted folder-name order, with digits preceding lowercase letters. The same class-to-index mapping must be maintained across training, validation, testing, and inference.

3.2 Image Preprocessing

Images are resized to 224 × 224 pixels. They are converted to tensors and normalized using the ImageNet channel statistics used by the pretrained models:

Mean: *[0.485, 0.456, 0.406]* ***Standard deviation:*** *[0.229, 0.224, 0.225]*

The data loaders use a batch size of 16 for the classification experiments. The training loader shuffles images, while validation and test loaders do not shuffle.

3.3 Data Augmentation

Three augmentation techniques are used to introduce variation into training images. Validation and test images use deterministic preprocessing without random augmentation.

Technique	Implementation	Purpose / note
Random horizontal flip	Probability 0.5	Introduces left-right variation. It should be used only if the flipped gesture retains the same class meaning.
Random rotation	Up to approximately ±10°	Helps the model tolerate small changes in hand orientation.
Color jitter	Brightness 0.2; contrast 0.2	Helps handle lighting and appearance variations.

3.4 Transfer Learning

Transfer learning adapts a model pretrained on a large image dataset to a new classification task. The pretrained models have learned general visual features. Their original output layers are replaced with layers containing 36 outputs for the ASL dataset.

ResNet18

ResNet18 uses residual connections that provide shortcut paths through the network. The pretrained model is loaded, its parameters are frozen, and the final fully connected layer is replaced with a 36-output layer. In the recorded setup, the new output layer is trainable while the pretrained backbone remains frozen.

VGG16

VGG16 uses a sequence of convolutional layers and pooling operations to learn increasingly complex image features. Its feature-extraction layers are frozen, and the final classifier layer is replaced with a 36-output layer.

AlexNet

AlexNet uses convolutional layers, activation functions, pooling, and fully connected layers for image classification. Its pretrained feature layers are frozen, and the final classification layer is adapted to produce 36 ASL class scores.

3.5 Custom CNN Architecture

The custom CNN learns features directly from the ASL dataset. It consists of two convolutional blocks followed by a classifier. Each convolutional block contains a convolution, ReLU activation, and max-pooling operation.

Layer	Configuration	Output shape per image
Input	RGB, 224 × 224	3 × 224 × 224
Convolution 1	3 → 16 channels; 3 × 3 kernel; padding 1	16 × 224 × 224
ReLU 1	Nonlinear activation	16 × 224 × 224
Max Pool 1	2 × 2; stride 2	16 × 112 × 112
Convolution 2	16 → 32 channels; 3 × 3 kernel; padding 1	32 × 112 × 112
ReLU 2	Nonlinear activation	32 × 112 × 112
Max Pool 2	2 × 2; stride 2	32 × 56 × 56
Flatten	Convert feature maps to vector	100,352
Fully connected 1	100,352 → 128	128
ReLU	Nonlinear activation	128
Fully connected 2	128 → 36	36 class scores

The convolution layers learn image patterns such as edges, curves, and hand shapes. Max pooling reduces spatial dimensions. Flatten converts the feature maps into a vector, and the fully connected layers map that vector to 36 class scores.

3.6 Custom CNN Parameter Calculation

For a convolutional layer, the number of parameters is (kernel height × kernel width × input channels × output channels) + output channels. For a fully connected layer, it is (input features × output features) + output features.

Layer	Calculation	Parameters
Convolution 1	(3 × 3 × 3 × 16) + 16	448
Convolution 2	(3 × 3 × 16 × 32) + 32	4,640
Fully connected 1	(100,352 × 128) + 128	12,845,184
Fully connected 2	(128 × 36) + 36	4,644
Total	Sum of trainable layer parameters	12,854,916

ReLU, max-pooling, and flattening layers do not introduce trainable parameters. The total of 12,854,916 agrees with the model inspection output recorded in the project documentation.

3.7 YOLO Hand Detection

YOLO (You Only Look Once) is an object-detection algorithm that predicts object locations and classes in an image. Unlike an image classifier, which assigns a class to an entire image, YOLO identifies object locations using bounding boxes.

The detector is configured as a single-class model with class identifier 0 corresponding to **hands**. The dataset is organized into images/train, images/val, labels/train, labels/val, and a data.yaml configuration file. Each annotation file describes the hand bounding box and its class identifier.

The best.pt checkpoint is selected for inference. The detected bounding box is used to crop the hand region, which is then prepared for the CNN classifier. Detection metrics such as precision, recall, mAP@0.5, and mAP@0.5:0.95 are used to evaluate the detector separately from classification accuracy.

CHAPTER 4 — IMPLEMENTATION AND TRAINING

4.1 Tools and Execution Environment

Technology	Purpose
Python	Main programming language
PyTorch	Neural-network implementation and training
Torchvision	Pretrained models, image transformations, and datasets
NumPy	Numerical operations and data handling
Pandas	Organizing and comparing experimental results
Scikit-learn	Stratified five-fold cross-validation
Kaggle Notebooks	Development and model-training environment
YOLO	Hand object detection

The notebook selected CUDA when available and otherwise selected the CPU. The recorded initial device check indicated CUDA. Actual training time can vary with GPU availability, model architecture, data loading, and runtime conditions.

4.2 Classification Training

The classification models were implemented using PyTorch and Torchvision. ImageFolder loads class-specific images, and DataLoader supplies batches during training. The training procedure consists of a forward pass, loss calculation using cross-entropy, backpropagation, and parameter updates using the Adam optimizer.

The pretrained models were trained for ten epochs in the recorded experiment. The original training loop was subsequently corrected to use the validation loader for epoch-wise evaluation rather than the test loader. The previously recorded transfer-learning results therefore remain preliminary and should be regenerated after this correction.

4.3 Custom CNN Training

The custom CNN uses the following recorded configuration:

Setting	Value
Input size	224 × 224 × 3
Number of classes	36
Batch size	16
Optimizer	Adam
Learning rate	0.001
Loss function	CrossEntropyLoss
Epochs	10
Best-model selection	Highest validation accuracy

During each training batch, the model enters training mode, gradients are cleared, images are passed through the network, cross-entropy loss is calculated, gradients are backpropagated, and the optimizer updates the trainable parameters. The best checkpoint is selected using validation accuracy and saved as a state_dict.

4.4 Five-Fold Cross-Validation

StratifiedKFold divides the selected training dataset into five folds while attempting to preserve class proportions. Each fold serves as the validation subset once; the remaining four folds are used for training. A fresh custom CNN is trained for each fold for ten epochs. The experiment uses a batch size of 32 and the 1,760 images in the training split.

The recorded implementation reused the training augmentation transformation for the validation subset within each fold. Consequently, validation images may also have been randomly augmented. A stricter evaluation would use deterministic validation transformations and apply augmentation only to the training fold.

4.5 YOLO Training and Integration

The YOLO workflow prepares image and annotation directories, verifies that class identifiers match the YAML configuration, trains the detector, evaluates it on validation images, and selects the best checkpoint. The supplied report describes the intended use of best.pt, but does not provide a complete final one-class YOLO evaluation table.

For integration, the YOLO checkpoint and CNN checkpoint are loaded separately. The input image or video frame is passed to YOLO; the detected hand bounding box is cropped; the crop is converted to RGB if necessary, resized to 224 × 224, and normalized using the classifier's preprocessing. The CNN predicts one of the 36 classes, and the output index is converted using the saved class mapping.

CHAPTER 5 — RESULTS AND DISCUSSION

5.1 Transfer-Learning Model Comparison

The following values are the recorded outputs from the original notebook run. Because that run used the test set during each training epoch, these values are preliminary and must be regenerated using validation data for epoch-wise evaluation and a reserved test set for final evaluation.

Model	Recorded accuracy	Recorded loss	Training time
ResNet18	93.94%	0.3388	134.40 s
VGG16	87.12%	0.4813	240.72 s
AlexNet	90.40%	0.2890	131.20 s

In the recorded run, ResNet18 had the highest reported accuracy (93.94%). AlexNet had the lowest recorded loss (0.2890) and the shortest recorded training time (131.20 seconds). VGG16 had the lowest recorded accuracy and the longest training time. These observations are preliminary rather than a final independent test comparison.

5.2 Custom CNN Results

Dataset	Loss	Accuracy
Training	0.1847	94.32%
Validation	0.2406	92.48%
Testing	0.1770	94.44%

The notebook's separate accuracy-check output recorded training accuracy as 94.09%, while the loss-and-accuracy evaluation recorded 94.32%. These are two recorded outputs from different evaluation passes. A single consistent evaluation should be run if one definitive training accuracy is required.

The custom CNN's recorded validation accuracy of 92.48% exceeds the project target of 83% by 9.48 percentage points. The recorded test accuracy is 94.44%. The test result is an estimate of generalization provided that the test split remained independent and was not used for model selection.

5.3 Five-Fold Cross-Validation Results

Fold	Best validation accuracy	Validation loss	Training time
1	90.91%	0.3556	98.98 s
2	88.35%	0.3430	99.41 s
3	90.63%	0.3987	100.12 s
4	90.34%	0.2856	99.59 s
5	90.06%	0.3664	101.63 s
Average	90.06%	0.3499	≈99.95 s

The best validation accuracy ranged from 88.35% to 90.91%. The mean best validation accuracy was approximately 90.06%, and the mean corresponding validation loss was approximately 0.3499. The fold results indicate reasonably consistent performance across the selected splits, subject to the augmentation caveat described in Chapter 4.

5.4 YOLO Detection Results

The hand detector was trained using YOLO11n, initialized with pretrained weights. The original gesture labels were merged into one detection class, ?hands?, because this stage locates the hand region; the separate CNN performs the 26-letter classification.

Training configuration

Epochs: 50 Image size: 640 × 640 Batch size: 16
Training images: 509 (503 labelled; 6 background) Validation images: 25 (24 labelled instances; 1 background)

Final validation metrics (best.pt)

Precision	0.958 (95.8%)
Recall	1.000 (100%)
mAP@0.5	0.969 (96.9%)
mAP@0.5:0.95	0.680 (68.0%)
Detector	YOLO11n · one class: hands
Checkpoint	runs/detect/asl_hands_detection/weights/best.pt

5.5 Discussion

The custom CNN achieved a recorded validation accuracy above the required 83% target. The architecture is relatively straightforward and provides a direct implementation of convolution, activation, pooling, flattening, and fully connected classification. The recorded cross-validation mean of 90.06% provides additional information across five splits, although the validation augmentation procedure should be improved for a stricter experiment.

The transfer-learning models reuse features learned from ImageNet and can be advantageous when the target dataset is comparatively small. However, the recorded transfer-learning comparison requires rerunning because the test set was used during epoch-wise evaluation. A fair final comparison should use the same data protocol for all models and reserve the test set for final evaluation.

YOLO and the CNN have separate responsibilities. YOLO must locate the hand accurately, while the CNN must classify the gesture from the crop. The quality of the complete system depends on both stages: an incorrect detection or unsuitable crop can reduce classification performance. Detection metrics and classification metrics should therefore be reported separately, and end-to-end performance should be measured only after the two models are connected and tested together.

Limitations

• The classification dataset is relatively small compared with large-scale image datasets.

• Gesture appearance varies with lighting, background, hand orientation, and camera quality.

• Some ASL classes may be visually similar.

• Horizontal flipping may not preserve the meaning of every gesture.

• Transfer-learning metrics need to be regenerated after correcting the validation loop.

• Cross-validation should use a separate deterministic validation transformation.

- YOLO validation used 25 images; the small validation set limits how broadly the reported metrics can be generalized.

• Static image classification does not fully represent dynamic signs that depend on motion or temporal information.

Integrated demonstration: the selected validation image B_photo_20.jpg produced one detected hand. The cropped region was passed to the CNN, which predicted B with 33.68% confidence. This is a single-image demonstration, not an end-to-end accuracy measurement.

CONCLUSION

This project investigates ASL hand-gesture recognition through image classification and hand detection. ResNet18, VGG16, and AlexNet are adapted using transfer learning, while a custom CNN is designed to learn features directly from the classification dataset. The classifier recognizes 36 classes consisting of digits 0–9 and lowercase letters a–z.

The custom CNN contains two convolutional blocks and two fully connected layers, with 12,854,916 trainable parameters. Its recorded validation accuracy of 92.48% exceeds the project's 83% target, and its recorded test accuracy is 94.44%. Five-fold cross-validation produced an average best validation accuracy of 90.06% in the recorded experiment.

The detection component uses YOLO to locate hands as a single object class. The intended integrated pipeline crops the detected hand and passes it to the CNN so that hand location and gesture meaning are determined by separate models. The final one-class YOLO metrics and complete end-to-end evaluation were not available in the supplied report.

Before final submission, the transfer-learning experiments should be rerun with the corrected validation procedure, the final YOLO evaluation values should be inserted, and the integrated detector-classifier pipeline should be tested on independent images or camera frames. Overall, the project demonstrates how transfer learning, custom CNN design, data augmentation, cross-validation, and object detection can be combined in an ASL gesture-recognition workflow.

REFERENCES

1. PyTorch Documentation. https://pytorch.org/docs/stable/index.html

2. Torchvision Documentation. https://pytorch.org/vision/stable/index.html

3. Scikit-learn Documentation. StratifiedKFold. https://scikit-learn.org/

4. Ultralytics Documentation. YOLO Documentation. https://docs.ultralytics.com/

5. Simonyan, K., and Zisserman, A. Very Deep Convolutional Networks for Large-Scale Image Recognition. 2014.

6. He, K., Zhang, X., Ren, S., and Sun, J. Deep Residual Learning for Image Recognition. 2015.

7. Krizhevsky, A., Sutskever, I., and Hinton, G. ImageNet Classification with Deep Convolutional Neural Networks. 2012.